



**UNIVERSIDAD
DE LA FRONTERA**

UNIVERSIDAD DE LA FRONTERA

Ingeniería Informática

Informe técnico

Taller de Redes: infraestructura de servicios para Fauna Araucanía

Grupo 3 – zorro-darwin.lazos.cl

Integrantes: Luis Jaramillo, Giovanni Toledo y Maximiliano Rivas

Profesor: Flavio Rojas Acuña

Fecha de entrega: 28 de septiembre de 2026

Índice

1. Introducción y objetivos	1
1.1. Objetivo general	1
1.2. Objetivos específicos	1
1.3. Cómo leer este informe	1
1.4. Alcance del trabajo	2
2. Infraestructura, arquitectura y plan de nombres	2
2.1. El VPS y su estado inicial – actividad 1	2
2.2. Diseño de la solución	3
2.3. Plan de nombres	3
3. Implementación y validación por actividad	4
3.1. Usuarios, directorios y permisos – actividad 2	4
3.2. MariaDB, PowerDNS y zona DNS – actividad 3	5
3.3. PowerAdmin – actividad 4	6
3.4. Apache, VirtualHosts y tres CMS – actividades 5 y 6	7
3.5. vsftpd – actividad 7	10
3.6. Correo: Postfix, Dovecot y RoundCube – actividad 8	12
3.7. Firewall y SELinux – actividades 9 y 10	13
3.8. Aplicación de sockets – actividad 11	14
4. Verificación integrada – actividad 12	15
5. Dificultades encontradas y cómo las resolvimos	16
6. Análisis de resultados y limitaciones	17
7. Conclusiones	18
8. Referencias y extractos de evidencia	18
8.1. Referencias	18
8.2. Extractos de las evidencias guardadas	18
A. Parámetros de configuración que determinan el comportamiento	19

Índice de figuras

1.	Página inicial de AlmaLinux	3
2.	Entorno del VPS y salida a Internet	3
3.	Arquitectura de servicios	4
4.	Permisos de la jaula y el sitio	5
5.	Consultas DNS desde el cliente	6
6.	Validación DNS desde el VPS	6
7.	Alta del TXT temporal	7
8.	Zona DNS final de once registros	7
9.	Cadena de solicitud web	8
10.	WordPress en web1	9
11.	Joomla en web2	9
12.	Grav en web3	10
13.	Propietarios y permisos web	10
14.	Canales FTP en modo pasivo	11
15.	Prueba automatizada de FTP	11
16.	Flujo de entrega y lectura de correo	12
17.	Mensaje de CMS1 recibido por CMS2	13
18.	Respuesta de CMS2 recibida por CMS1	13
19.	Puertos autorizados en firewalld	14
20.	Dos sesiones de chat TCP	15
21.	Ciclo de conexión del chat	15

Índice de cuadros

1.	Niveles de comprobación que aplicamos a cada servicio.	1
2.	Nombres DNS y destinos de los servicios.	4
3.	Cuentas, bases de datos y nomenclatura aplicada.	4
4.	CMS, persistencia y contenido publicado.	8
5.	Puertos publicados y su justificación.	14
6.	Resultados de la verificación integrada.	16
7.	Dificultades: síntoma, diagnóstico, solución y resultado.	16
8.	Parámetros clave de cada servicio y su motivo.	19

1. Introducción y objetivos

Fauna Araucanía es una empresa que quiere tener presencia en Internet, pero hoy solo cuenta con un servidor VPS y un dominio, sin DNS propio ni ningún servicio de red funcionando. Nuestro trabajo, como equipo de asesores, fue convertir ese servidor vacío en una infraestructura completa: nombres de dominio, tres sitios web, correo electrónico, transferencia de archivos y una aplicación cliente-servidor.

Trabajamos sobre un VPS con AlmaLinux, IP 200.13.5.39 y el dominio `zorro-darwin.lazos.cl`. Este informe cuenta qué instalamos, cómo lo configuramos, por qué tomamos cada decisión y cómo comprobamos que funciona. Cuando algo falló, lo explicamos también, porque de esos errores aprendimos más que de las instalaciones que salieron bien.

1.1. Objetivo general

Implementar, asegurar, documentar y validar una infraestructura integrada de servicios de red en un VPS Linux, que permita a Fauna Araucanía resolver nombres DNS, alojar tres sitios web, usar correo electrónico, transferir archivos y ejecutar una aplicación de sockets.

1.2. Objetivos específicos

1. Reconocer y documentar el estado inicial del VPS antes de instalar nada.
2. Crear tres cuentas locales, una por sitio, con permisos separados y sin recurrir a 777.
3. Montar un DNS autoritativo con PowerDNS y administrarlo con PowerAdmin.
4. Publicar tres sitios en Apache, cada uno con un CMS distinto (WordPress, Joomla y Grav).
5. Configurar vsftpd con puerto alternativo, sin acceso anónimo y con cada usuario encerrado en su directorio.
6. Implementar correo con Postfix, Dovecot y RoundCube, y comprobar el envío y la recepción.
7. Abrir en el firewall solo los puertos necesarios y mantener SELinux en modo **Enforcing**.
8. Poner en marcha el servidor de sockets y probarlo desde un equipo externo.
9. Validar todo en conjunto y documentar los problemas que encontramos.

1.3. Cómo leer este informe

Para cada actividad intentamos responder siempre las mismas preguntas: *qué hicimos*, *cómo lo configuramos*, *por qué lo decidimos así*, *cómo lo comprobamos* y *qué resultado obtuvimos*. No incluimos una captura por cada comando; agrupamos los pasos relacionados y mostramos capturas representativas del resultado, cada una con su explicación.

Una idea guía atraviesa todo el trabajo: que un servicio aparezca como “activo” no prueba que funcione. Por eso, además de revisar su estado, lo probamos como lo haría un usuario real (desde otro equipo, enviando un correo, subiendo un archivo). La Tabla 1 resume esos tres niveles de comprobación.

Cuadro 1: Niveles de comprobación que aplicamos a cada servicio.

Nivel	Herramientas	Qué nos permite saber
Configuración	<code>stat</code> , <code>getsebool</code> , <code>firewall-cmd</code> , <code>httpd -S</code>	Si propietarios, políticas, puertos y rutas están como los diseñamos.
Estado	<code>systemctl is-active</code> , <code>ss</code> , <code>apachectl configtest</code>	Si el servicio está corriendo y su configuración es válida.
Función	<code>dig</code> , <code>curl</code> , cliente FTP, RoundCube, cliente Python	Si un usuario, desde otro equipo, obtiene el resultado esperado.

1.4. Alcance del trabajo

El alcance de este informe son los servicios pedidos por el taller: DNS con PowerDNS y PowerAdmin, tres sitios con tres CMS, FTP con jaulas, correo con Postfix, Dovecot y RoundCube, firewall, SELinux y la aplicación de sockets. Para el correo usamos IMAP e IMAPS, y los sitios y el webmail se publican por HTTP, que es lo que necesitábamos para demostrar el funcionamiento en el entorno del taller. Más adelante explicamos qué habría que cambiar para un uso real.

En la consulta final por SSH al VPS (archivo `evidencias/11-validacion-final/consulta-ssh-20260929.txt`) las nueve unidades revisadas estaban activas, SELinux informó **Enforcing**, los sitios y el webmail respondieron HTTP 200 y PowerAdmin respondió HTTP 302 (la redirección a su pantalla de acceso). En esa sesión no pudimos usar `sudo`, así que no repetimos las consultas que requieren privilegios (zona y firewall); esas se hicieron en sesiones anteriores.

2. Infraestructura, arquitectura y plan de nombres

2.1. El VPS y su estado inicial – actividad 1

Qué hicimos. Antes de instalar cualquier cosa, revisamos cómo venía el servidor, para poder comparar después. El 26 de septiembre consultamos la distribución, los recursos, la red, SELinux, el firewall y los servicios que ya corrían.

Qué encontramos. El VPS usa AlmaLinux 9.8, con la interfaz `ens18`, IP `200.13.5.39`, 3.6 GiB de RAM, 3.2 GiB de swap y 28 GiB de disco en la raíz. El nombre de host es `localhost.localdomain` y la zona horaria `Europe/Madrid`. SELinux ya venía en **Enforcing**; `firewalld` permitía SSH, HTTP, Cockpit y DHCPv6, y Apache mostraba su página de prueba, sin ningún sitio nuestro.

Decisión. No cambiamos el `hostname`: no era necesario para que los VirtualHosts, el DNS ni el correo funcionaran, y evitamos así tocar algo que podía afectar otros servicios.

Evidencia. La Figura 1 muestra la página de prueba de Apache, que confirma que el servicio web existía pero aún no alojaba nada nuestro. La Figura 2 muestra, desde el propio VPS, la versión de AlmaLinux, la interfaz de red con su IP, SELinux en **Enforcing** y una solicitud HTTP saliente que respondió HTTP/2 200, lo que confirma que el servidor tiene salida a Internet.

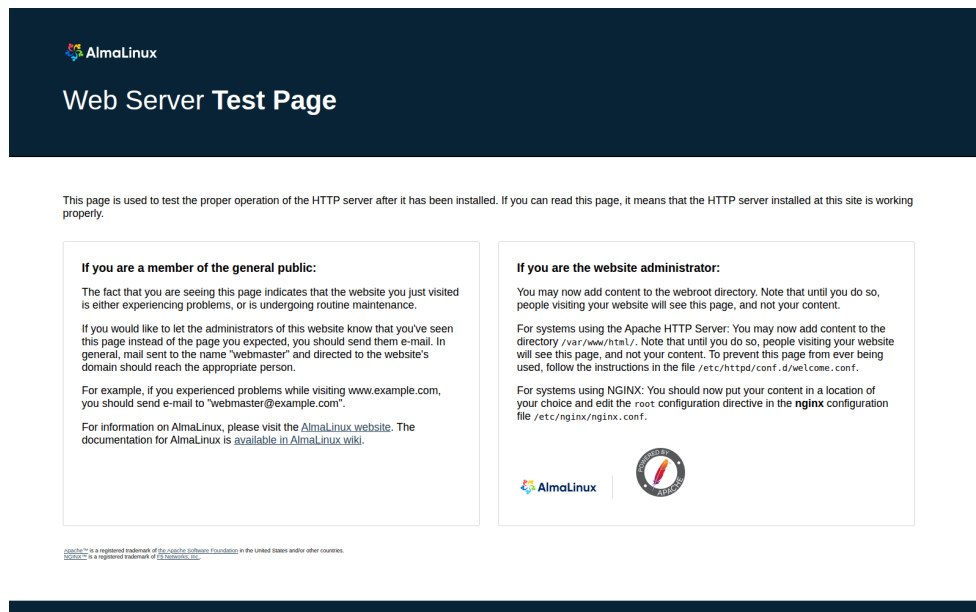


Figura 1: Página de prueba de AlmaLinux antes del despliegue. Sirve como referencia del estado inicial: el servicio HTTP existía, pero todavía no había un sitio del taller.

```
$ ssh -F /dev/null -o BatchMode=yes -i /home/gtoledo/Descargas/tredes3.key tredes3@200.13.5.39 'hostname; . /etc/os-release; echo "${PRETTY_NAME}"; ip -br -4 addr show ens18; getenforce; curl -sSI --max-time 8 https://example.com | head -1'
localhost.localdomain
AlmaLinux 9.8 (Olive Jaguar)
ens18 UP 200.13.5.39/26
Enforcing
HTTP/2 200
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
[salida 0]
```

Figura 2: AlmaLinux 9.8, interfaz ens18 con 200.13.5.39/26, SELinux en Enforcing y una respuesta HTTP/2 200 a una solicitud saliente. Capturada después del despliegue.

2.2. Diseño de la solución

Decidimos concentrar todos los servicios en el mismo VPS y mantener sus datos ordenados y separados. La Figura 3 muestra cómo se relacionan las piezas. En pocas palabras:

- **MariaDB** guarda los datos de PowerDNS, WordPress, Joomla y PowerAdmin, y solo acepta conexiones locales (127.0.0.1).
- **Apache** recibe todas las peticiones web en el puerto 80 y elige el sitio según el nombre que pidió el visitante.
- **Postfix, Dovecot y RoundCube** cubren el envío, el almacenamiento y la lectura del correo.
- **vsftpd** reutiliza las cuentas de los sitios para que cada usuario administre solo sus archivos.
- **El chat** (aplicación de sockets) escucha en el puerto TCP/9000.

Reutilizamos las cuentas locales para FTP y para los buzones de correo, pero mantuvimos la separación mediante propietarios, permisos Unix y políticas SELinux: vsftpd encierra cada sesión en su HOME, Apache lee `public_html`, Postfix entrega en formato Maildir y Dovecot expone esos buzones a RoundCube.

2.3. Plan de nombres

Todos los nombres apuntan a la misma IP del VPS, porque todos los servicios viven en un solo servidor. La Tabla 2 lista los nombres y su función. Apache distingue los sitios por el encabezado `Host`, no por la IP.

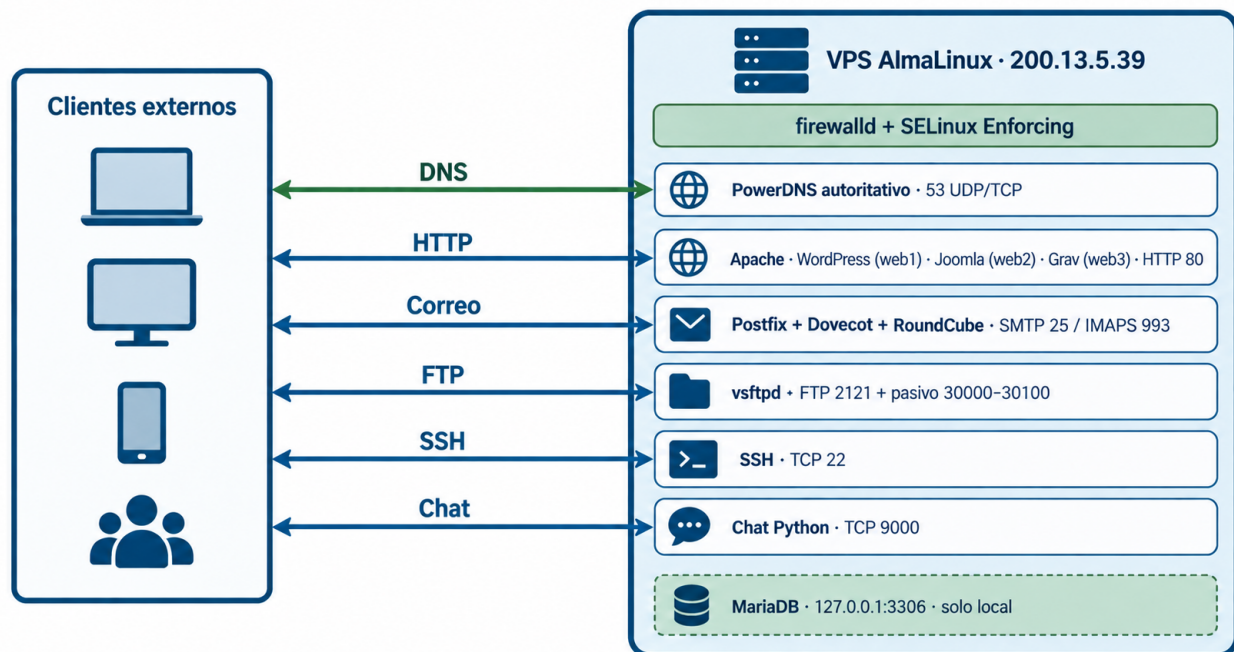


Figura 3: Arquitectura del despliegue. Todos los servicios corren en el mismo VPS y MariaDB solo acepta conexiones locales. Es un diagrama de diseño, no una captura de prueba.

Cuadro 2: Nombres DNS y destinos de los servicios.

Nombre	Registros / destino	Servicio
raíz	A 200.13.5.39; MX 10 mail; NS ns1; SOA	Zona y correo del dominio
ns1	A 200.13.5.39	DNS autoritativo
web1 / web2 / web3	A 200.13.5.39 cada uno	WordPress / Joomla / Grav
mail / webmail	A 200.13.5.39 cada uno	Postfix-Dovecot / RoundCube
dnsadmin	A 200.13.5.39	PowerAdmin

La zona final tiene 11 registros y `pdnsutil zone check` no reportó errores ni avisos. Un detalle que conviene aclarar: la zona padre delega el dominio en `zorro-darwin-ns.lazos.cl`, mientras que dentro de nuestra zona declaramos `ns1.zorro-darwin.lazos.cl`. Ambos nombres resuelven a `200.13.5.39`. La delegación del padre la administra el curso y nosotros no la modificamos; nos encargamos de que ambos nombres resolvieran a nuestro VPS.

3. Implementación y validación por actividad

3.1. Usuarios, directorios y permisos – actividad 2

Qué hicimos. Creamos las tres cuentas locales pedidas, una por sitio: `tresdes3-cms1`, `tresdes3-cms2` y `tresdes3-cms3`. La cuenta `tresdes3`, entregada por el curso, la usamos para entrar por SSH y ejecutar el chat; no cuenta como cuenta de sitio. Las contraseñas siguen el formato `CmsX_tRedeSN_WXYZ` y no se reproducen aquí.

Cuadro 3: Cuentas, bases de datos y nomenclatura aplicada.

Servicio	Cuenta local	Base / usuario SQL	Contraseña
WordPress	<code>tresdes3-cms1</code>	<code>web1_db / web1_user</code>	Formato requerido; omitida
Joomla	<code>tresdes3-cms2</code>	<code>web2_db / web2_user</code>	Formato requerido;

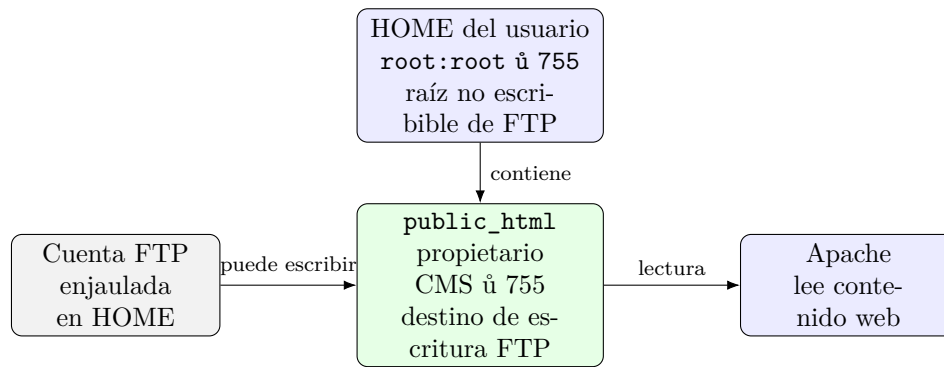


Figura 4: Separación de permisos de cada sitio. El HOME, propiedad de root, fija la jaula; la cuenta del CMS modifica su `public_html`, que Apache lee.

Resultado. Tres identidades independientes, con directorios propios y permisos ajustados, base para el aislamiento web y FTP que se prueba más adelante.

3.2. MariaDB, PowerDNS y zona DNS – actividad 3

Qué hicimos. Instalamos PowerDNS como servidor DNS *autoritativo* (responde con los datos de nuestra zona, a diferencia de un resolvidor recursivo, que busca respuestas para otros) y le agregamos la zona `zorro-darwin.lazos.cl`.

Cómo lo configuramos. PowerDNS guarda sus registros en MariaDB mediante el backend `gmysql`. Creamos la base `pdns_redes3` y el usuario `dominio_pdns` con permisos limitados solo a esa base, y configuramos MariaDB en `127.0.0.1:3306`. En la zona usamos cuatro tipos de registro: SOA (quién es la autoridad y cada cuánto se actualiza), NS (qué servidor responde), A (dirección IPv4 de cada nombre) y MX (adónde va el correo).

Por qué esas decisiones. Usamos MariaDB porque permite que PowerDNS y PowerAdmin compartan los mismos datos, y la mantuvimos en loopback para no exponer la base a Internet. PowerAdmin, que corre dentro de Apache, necesita conectarse a esa base; para permitirlo activamos el booleano SELinux `httpd_can_network_connect_db` en lugar de desactivar SELinux.

Un error que nos enseñó algo. Al principio la zona no tenía el registro A de la raíz. `pdnsutil zone check` igual daba resultado válido, porque revisa la sintaxis de lo que existe, no lo que falta; pero al consultar el nombre principal no obtuvimos dirección. Agregamos el registro y la zona quedó en 11 registros, con cero errores y cero avisos.

Evidencia. Consultamos la zona desde un equipo externo, directamente al servidor (200.13.5.39) por UDP y TCP y también a través del resolvidor público 1.1.1.1. Las respuestas directas traen la marca `aa` (respuesta autoritativa) y la consulta pública devolvió el A de la raíz. La Figura 5 muestra el resultado de SOA, NS, A y MX; la Figura 6 muestra la misma información vista desde el propio VPS por SSH (PowerDNS y MariaDB activos, DNS escuchando, SOA con serial 2026092801 y MX). Comparar ambos puntos de vista permite confirmar que lo almacenado en la base es lo que efectivamente se responde en la red.

```

+ date -Is
2026-09-28T17:35:03-03:00
+ hostname
home
+ dig @dell.ns.cloudflare.com +norecurse +noall +authority +additional zorro-darwin.lazos.cl NS
zorro-darwin.lazos.cl. 300 IN NS zorro-darwin.ns.lazos.cl.
zorro-darwin.ns.lazos.cl. 300 IN A 200.13.5.39
+ dig @200.13.5.39 +norecurse +noall +answer zorro-darwin.lazos.cl NS
zorro-darwin.lazos.cl. 3600 IN NS ns1.zorro-darwin.lazos.cl.
+ dig @200.13.5.39 +norecurse +noall +answer zorro-darwin.lazos.cl SOA
zorro-darwin.lazos.cl. 3600 IN SOA ns1.zorro-darwin.lazos.cl. hostmaster.zorro-darwin.lazos.cl. 2026092801 10800 3600
604800 3600
+ dig @200.13.5.39 +norecurse +noall +answer zorro-darwin.lazos.cl A
zorro-darwin.lazos.cl. 300 IN A 200.13.5.39
+ dig @1.1.1.1 +noall +answer zorro-darwin.lazos.cl A
zorro-darwin.lazos.cl. 300 IN A 200.13.5.39
+ dig @200.13.5.39 +norecurse +noall +answer zorro-darwin.lazos.cl MX
zorro-darwin.lazos.cl. 3600 IN MX 10 mail.zorro-darwin.lazos.cl.
+ dig @200.13.5.39 +norecurse +noall +answer mail.zorro-darwin.lazos.cl A
mail.zorro-darwin.lazos.cl. 3600 IN A 200.13.5.39
+ dig @200.13.5.39 +tcp +norecurse +noall +answer zorro-darwin.lazos.cl A
zorro-darwin.lazos.cl. 300 IN A 200.13.5.39

```

Figura 5: Consultas del 28-09-2026 desde un equipo externo: NS del padre y de la zona, SOA, A de la raíz y MX hacia mail. Las consultas por UDP, TCP y 1.1.1.1 devolvieron lo esperado.

```

active
udp UNCONN 0 0 0.0.0.0:53 0.0.0.0:*
tcp LISTEN 0 128 0.0.0.0:53 0.0.0.0:*
tcp LISTEN 0 80 127.0.0.1:3306 0.0.0.0:*
zorro-darwin.lazos.cl. 3600 IN SOA ns1.zorro-darwin.lazos.cl. hostmaster.zorro-darwin.lazos.cl. 2026092801 10800 3600
604800 3600
zorro-darwin.lazos.cl. 3600 IN MX 10 mail.zorro-darwin.lazos.cl.

```

Figura 6: Salida remota de la sesión SSH del 28-09-2026: PowerDNS y MariaDB activos, escucha DNS, SOA y MX. Se omitió el comando local para no mostrar la ruta de la llave SSH.

Resultado. La zona responde correctamente desde fuera del VPS, con autoridad, por UDP y TCP, y el MX apunta a mail.

3.3. PowerAdmin – actividad 4

Qué hicimos. Instalamos PowerAdmin, la interfaz web para ver y editar los registros DNS, y la publicamos como `dnsadmin.zorro-darwin.lazos.cl`, conectada a la base `pdns_redes3`.

Cómo lo comprobamos. No nos bastó con que la página cargara. Hicimos una prueba de escritura real: respaldamos la base, creamos una cuenta temporal, agregamos un registro TXT desde la interfaz, lo consultamos con `dig` y luego lo borramos (junto con la cuenta temporal) para no dejar datos de prueba en la zona entregada. Así demostramos que un cambio hecho en la web llega a lo que el servidor DNS responde.

Un problema y su explicación. Justo después de crear el TXT, `dig` no lo mostraba: la caché de PowerDNS conservaba una respuesta negativa anterior. Purgamos solo ese nombre con `pdns_control` y el dato apareció; al borrarlo, ocurrió lo mismo a la inversa. También, tras instalar, PowerAdmin mostraba un error porque el directorio del instalador seguía presente; lo movimos al respaldo.

Evidencia. La Figura 7 muestra el TXT temporal creado en PowerAdmin y la Figura 8 la zona final con los 11 registros que coinciden con las consultas `dig` de la sección anterior.

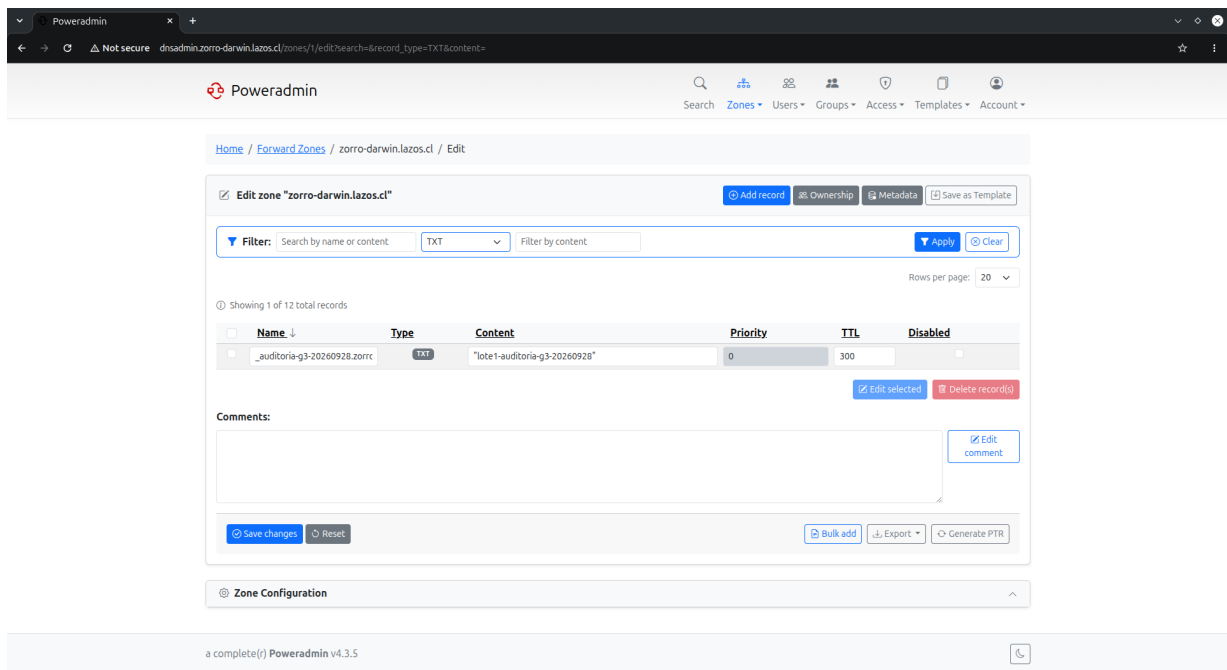


Figura 7: PowerAdmin autenticado, con la URL visible: TXT temporal creado (zona con 12 registros en ese momento). La consulta autoritativa, tras purgar la caché, devolvió ese valor.

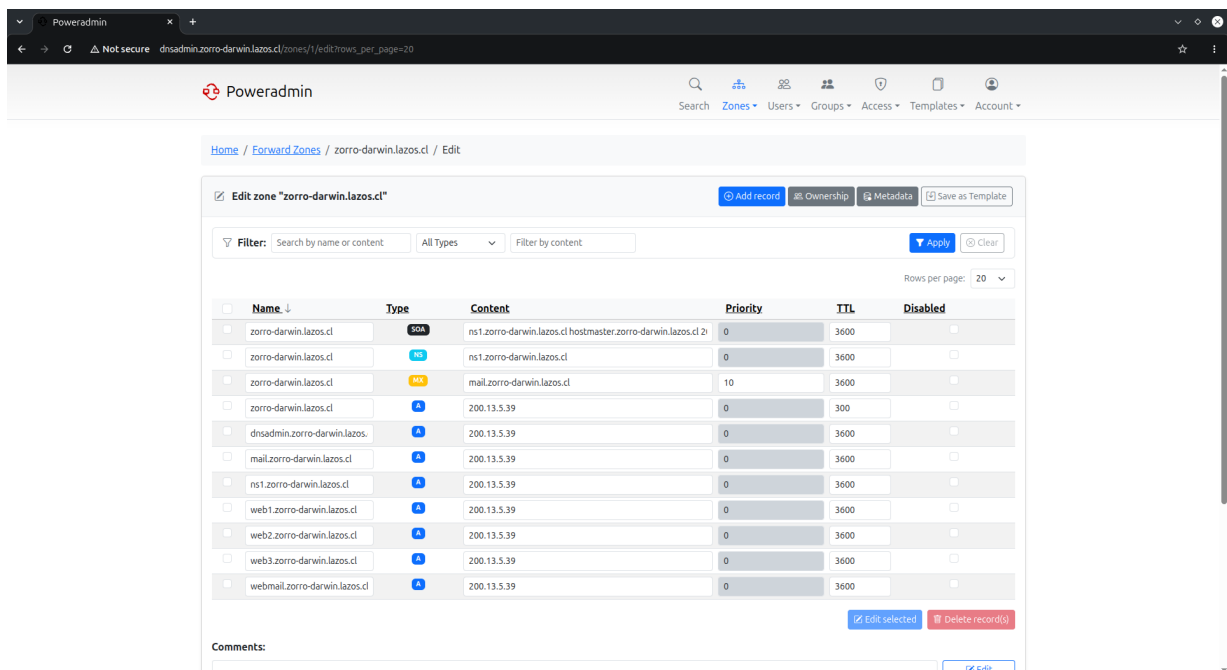


Figura 8: Zona final en PowerAdmin, con los 11 registros operativos después de borrar el TXT temporal. La ausencia del TXT también se verificó con dig.

Resultado. PowerAdmin es accesible por nombre, muestra la zona completa y sus cambios se reflejan en el DNS real.

3.4. Apache, VirtualHosts y tres CMS – actividades 5 y 6

Qué hicimos. Instalamos Apache 2.4 con PHP 8.3 (PHP-FPM) y publicamos tres sitios, cada uno con un CMS distinto: WordPress en web1, Joomla en web2 y Grav en web3. Los tres VirtualHosts

están en `/etc/httpd/conf.d/cms-sites.conf`, cada uno con su propio DocumentRoot dentro del `public_html` de su cuenta.

Cómo funciona. Los tres nombres apuntan a la misma IP. Cuando alguien abre `web2.zorro-darwin.lazos.cl`, el DNS entrega la IP, el navegador envía la petición con ese nombre en el encabezado `Host`, Apache lo usa para elegir el VirtualHost correcto y PHP-FPM ejecuta el CMS. WordPress y Joomla consultan MariaDB; Grav no necesita base de datos porque guarda sus páginas como archivos Markdown (la base `web3_db` existe pero quedó vacía y no se usa). La Figura 9 resume este recorrido.

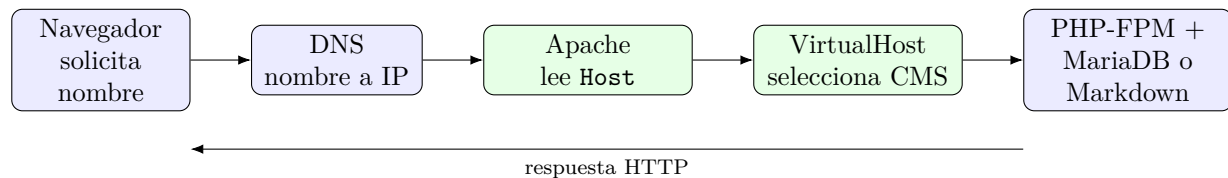


Figura 9: Recorrido de una solicitud web. WordPress y Joomla usan MariaDB; Grav lee archivos Markdown.

Cuadro 4: CMS, persistencia y contenido publicado.

Sitio	CMS y persistencia		Contenido final	Prueba
web1	WordPress	7.1.2; <code>web1_db</code> , prefijo <code>wp3_</code> ; cuenta CMS1	Artículo del zorro de Darwin	Portada HTTP 200; escritura acotada en uploads
web2	Joomla	6.1.3; <code>web2_db</code> , prefijo <code>g3j_</code> ; cuenta CMS2	Artículo de bosque nativo	Artículo guardado y publicado
web3	Grav	2.2.1; archivo Markdown; cuenta CMS3	Guía de hábitats	Portada HTTP 200

Decisiones sobre permisos y SELinux. Como Apache solo debe escribir donde el CMS lo necesita, en WordPress limitamos su escritura a la carpeta `uploads` con una ACL y una etiqueta SELinux específica, y en Joomla etiquetamos solo las rutas de caché necesarias. El contenido que solo se lee lleva la etiqueta `httpd_sys_content_t` y el que se escribe, `httpd_sys_rw_content_t`. Activamos `httpd_enable_homedirs` para que Apache pueda servir desde los HOME. Las etiquetas se hicieron persistentes con reglas de contexto y se verificaron con `restorecon` y `matchpathcon -V`, para que sobrevivan a un reetiquetado.

Problemas encontrados. Joomla devolvía HTTP 500 porque el límite de memoria de PHP era insuficiente y su caché no tenía la etiqueta SELinux de escritura; subimos el límite a 512 MiB y etiquetamos esa ruta. Además, la primera edición del artículo en Joomla mostraba etiquetas HTML como texto; lo corregimos desde su administrador y comprobamos el registro en la base (`g3j_content`). Grav guarda su guía en `/home/tredes3-cms3/public_html/user/pages/01.home/default.md`.

Evidencia. Antes de publicar el DNS usamos `curl --resolve` para enviar el `Host` de cada sitio directamente al VPS, lo que permite probar los VirtualHosts sin depender del DNS. Después consultamos los nombres públicos. `apachectl configtest` validó la sintaxis y `httpd -S` mostró los nombres asignados a cada archivo de sitio. Las Figuras 10, 11 y 12 muestran cada portada con su contenido propio, y la Figura 13 muestra propietarios y modos de cada `public_html`.

Blog

Zorro de Darwin: patrimonio del bosque templado

Sitio demostrativo de Fauna Araucanía, implementado por el Grupo 3 en WordPress.

Esta sección publica noticias sobre fauna nativa y conservación. El zorro de Darwin identifica nuestro dominio de trabajo: zorro-darwin.lazos.cl.

Prueba de infraestructura: web1 utiliza una cuenta local y una base de datos independientes de Joomla y Grav.

September 26, 2026

Fauna Araucanía | Noticias — Grupo 3

CMS 1 · WordPress · Taller de Redes

[Blog](#)

[About](#)

[FAQs](#)

[Authors](#)

[Events](#)

[Shop](#)

[Patterns](#)

[Themes](#)

Twenty Twenty-Five

Designed with [WordPress](#)

Figura 10: web1 muestra el artículo sobre el zorro de Darwin publicado con WordPress.

CASSIOPEIA

You are here: [Home](#)

Home

[Conservación del bosque nativo en La Araucanía](#)

Details
 Written by: [Administrador Grupo 3](#)
 Category: [Uncategorised](#)
 Published: 27 September 2026
 Hits: 0

Fauna Araucanía presenta este espacio de divulgación construido con Joomla para el Taller de Redes del Grupo 3. El bosque templado es hábitat del zorro de Darwin y otras especies nativas. Este sitio demuestra publicación independiente, administración propia y resolución mediante [web2.zorro-darwin.lazos.cl](#).

Main Menu

- [Home](#)

Login Form

Username

Password

☐ Remember Me

[Log in](#)

[Forgot your password?](#)
[Forgot your username?](#)

Figura 11: web2 muestra el artículo de bosque nativo publicado con Joomla, ya corregido el problema de las etiquetas HTML.



Figura 12: web3 muestra la guía de hábitats que Grav carga desde su archivo Markdown.

```
$ stat -c '%U:%G %a %n' /home/tredes3-cms{1,2,3}/public_html
tredes3-cms1:tredes3-cms1 755 /home/tredes3-cms1/public_html
tredes3-cms2:tredes3-cms2 755 /home/tredes3-cms2/public_html
tredes3-cms3:tredes3-cms3 755 /home/tredes3-cms3/public_html
```

Figura 13: Propietarios y modos de los tres directorios `public_html`. La captura completa está en la carpeta de evidencias.

Resultado. Tres sitios independientes, con tres CMS distintos y contenido diferenciable, cada uno bajo su propio usuario y con permisos ajustados.

3.5. vsftpd – actividad 7

Qué hicimos. Configuramos vsftpd 3.0.5 para que cada una de las tres cuentas pueda administrar por FTP solo su propio sitio.

Conceptos que explican la configuración. FTP usa dos canales: uno de control (órdenes y respuestas) y otro de datos (los archivos). En modo activo el servidor abre la conexión de datos hacia el cliente; en modo pasivo el cliente abre ambas. Elegimos el **modo pasivo** porque suele funcionar mejor a través de firewalls y NAT, y porque el servidor puede anunciar exactamente qué puertos usará. La Figura 14 muestra ambos canales.

Cómo lo configuramos. Los parámetros clave son: control en el puerto 2121/TCP (el puerto 21 está bloqueado en la red del taller), rango pasivo 30000-30100/TCP anunciando la IP 200.13.5.39, acceso anónimo deshabilitado, solo las tres cuentas CMS autorizadas y `chroot_local_user=YES` con raíz en el HOME de cada usuario. Ese HOME es `root:root 755` (explicado en la actividad 2), y

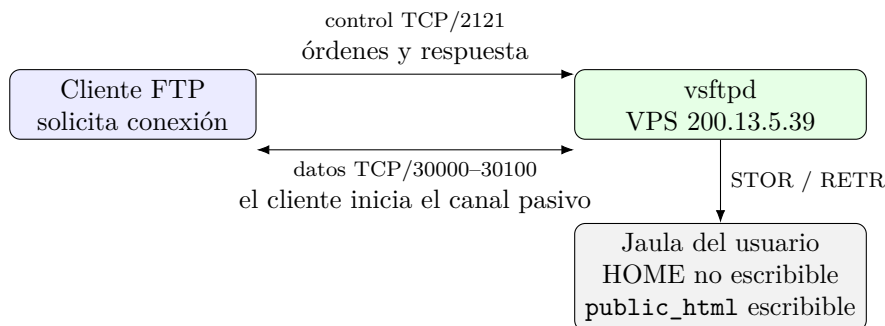


Figura 14: FTP pasivo: control en el puerto 2121 y datos en el rango 30000–30100. Las operaciones de archivo quedan dentro de la jaula de cada cuenta.

cada usuario escribe en su `public_html`; CMS1 y CMS2 además ven su Maildir. Abrimos esos puertos en `firewalld` y etiquetamos en SELinux el puerto de control como `ftp_port_t` y el rango de datos como `ftp_data_port_t`, además de activar `ftpd_use_passive_mode`. Una regla `tmpfiles` recrea `/run/vsftpd/empty`, necesaria porque `/run` es un directorio temporal.

El problema más instructivo: el error 553. Al subir un archivo, el login funcionaba pero la transferencia devolvía *553 Could not create file*, aunque los permisos Unix (DAC) del propietario permitían escribir. La causa estaba en SELinux: el proceso de `vsftpd` (`ftpd_t`) no tenía permiso para crear archivos en el tipo de contenido web de Apache. Lo diagnosticamos con `stat`, `ls -Z` y `seselect` (instalando `setools-console`) y lo resolvimos con un módulo de política local y acotado, `ftpd_httpd_content` (código en `configuraciones/selinux/ftpd_httpd_content.te`). No activamos `ftpd_full_access` ni `ftpd_anon_write`, y SELinux siguió en `Enforcing`. La lección: los permisos de archivo y la política SELinux son dos controles distintos, y ambos deben permitir la operación.

Evidencia. Probamos las tres cuentas con un cliente automatizado desde el equipo local. Cada cuenta se autenticó, listó su jaula, subió un archivo y lo reemplazó (respuesta 226 en cada transferencia), y el sitio web correspondiente devolvió HTTP 200 con el contenido idéntico al subido. Esto demuestra la correspondencia usuario, sitio y directorio FTP. El intento anónimo recibió 530, y los intentos de entrar a `/etc` o al `HOME` de otra cuenta recibieron 550, porque esas rutas simplemente no existen dentro de la jaula. La Figura 15 resume la prueba.

```

tresdes3-cms1 autenticado, raiz= /
Aislamiento /etc → 550 Failed to change directory.
Aislamiento /home/tresdes3-cms2 → 550 Failed to change directory.
Subida prueba-grupo3-cms1.txt versión 1 → 226 Transfer complete.
HTTP 200 contenido coincide: True Grupo 3 - CMS 1 - version 1
Subida prueba-grupo3-cms1.txt versión 2 → 226 Transfer complete.
HTTP 200 contenido coincide: True Grupo 3 - CMS 1 - version 2
tresdes3-cms2 autenticado, raiz= /
Aislamiento /etc → 550 Failed to change directory.
Aislamiento /home/tresdes3-cms3 → 550 Failed to change directory.
Subida prueba-grupo3-cms2.txt versión 1 → 226 Transfer complete.
HTTP 200 contenido coincide: True Grupo 3 - CMS 2 - version 1
Subida prueba-grupo3-cms2.txt versión 2 → 226 Transfer complete.
HTTP 200 contenido coincide: True Grupo 3 - CMS 2 - version 2
tresdes3-cms3 autenticado, raiz= /
Aislamiento /etc → 550 Failed to change directory.
Aislamiento /home/tresdes3-cms1 → 550 Failed to change directory.
Subida prueba-grupo3-cms3.txt versión 1 → 226 Transfer complete.
HTTP 200 contenido coincide: True Grupo 3 - CMS 3 - version 1
Subida prueba-grupo3-cms3.txt versión 2 → 226 Transfer complete.
HTTP 200 contenido coincide: True Grupo 3 - CMS 3 - version 2
Anónimo rechazado: 530 Permission denied.
  
```

Figura 15: Las tres cuentas transfirieron archivos y el cliente comparó su contenido por HTTP. La transcripción completa registra también el rechazo del acceso anónimo.

Resultado. FTP en puerto alternativo, sin acceso anónimo, con cada usuario aislado y escribiendo solo en su sitio, todo con SELinux activo. Esto demuestra el aislamiento en los casos probados; no afirmamos haber probado todas las rutas posibles.

3.6. Correo: Postfix, Dovecot y RoundCube – actividad 8

Cómo funciona el correo en nuestra solución. El registro MX indica qué servidor recibe el correo del dominio (mail). Postfix, el servidor SMTP, recibe cada mensaje y lo deposita en el buzón Maildir de la cuenta local. Dovecot permite leer ese buzón por IMAP, y RoundCube es la página web desde donde el usuario redacta y lee. La Figura 16 muestra el recorrido que probamos con las cuentas CMS1 y CMS2.

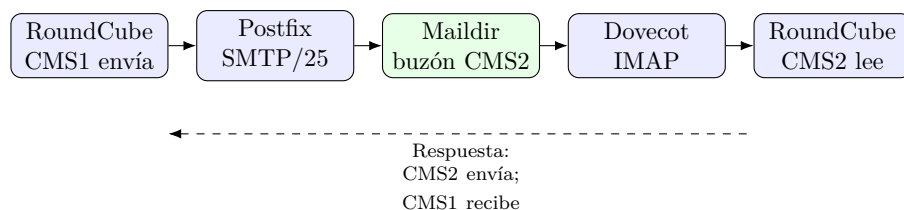


Figura 16: Flujo de correo probado: RoundCube entrega por SMTP a Postfix, que deposita el mensaje en Maildir; Dovecot expone el buzón por IMAP. La respuesta hace el mismo recorrido con las cuentas intercambiadas.

Qué configuramos. Postfix 3.5.25 en TCP/25, Dovecot 2.3.16 con IMAP e IMAPS (993) y RoundCube 1.7.4 publicado como **webmail**. Usamos un certificado autofirmado para el TLS del correo. Los parámetros clave están en la Tabla 8 del anexo.

Decisiones y límites. Usamos IMAP/IMAPS y **no implementamos POP3**. Tampoco publicamos HTTPS en el puerto 443: el webmail funciona por HTTP en este despliegue, lo que es aceptable para el laboratorio pero no para un servicio real.

Problemas encontrados. Hubo dos, y ambos se resolvieron leyendo los registros en lugar de probar cambios al azar:

1. “*User unknown in virtual alias table*”. Postfix trataba los destinatarios como alias virtuales. Configuramos el dominio en **mydestination** y usamos las cuentas locales.
2. “*Error SMTP*” en *RoundCube*. El registro mostraba *Connection refused*: RoundCube intentaba conectar a **localhost:587**, pero nuestro Postfix escuchaba en el 25. Fijamos **smtp_host=127.0.0.1:25** y revisamos la sintaxis con **php -l**.

Además, RoundCube necesitaba conectar a IMAP y SMTP locales bajo SELinux. En vez de activar **httpd_can_network_connect** (que da conectividad de red mucho más amplia), añadimos el módulo acotado **roundcube_http_mail**.

Evidencia. CMS1 envió un mensaje a CMS2 (Figura 17) y CMS2 respondió a CMS1 (Figura 18); en cada captura se ve el mensaje abierto en RoundCube en el buzón del destinatario, con remitente, asunto y cuerpo. Estas pruebas se ejecutaron con un navegador automatizado a través de un túnel SSH.



Figura 17: Mensaje de CMS1 abierto en el buzón de CMS2, tras corregir el destino SMTP.

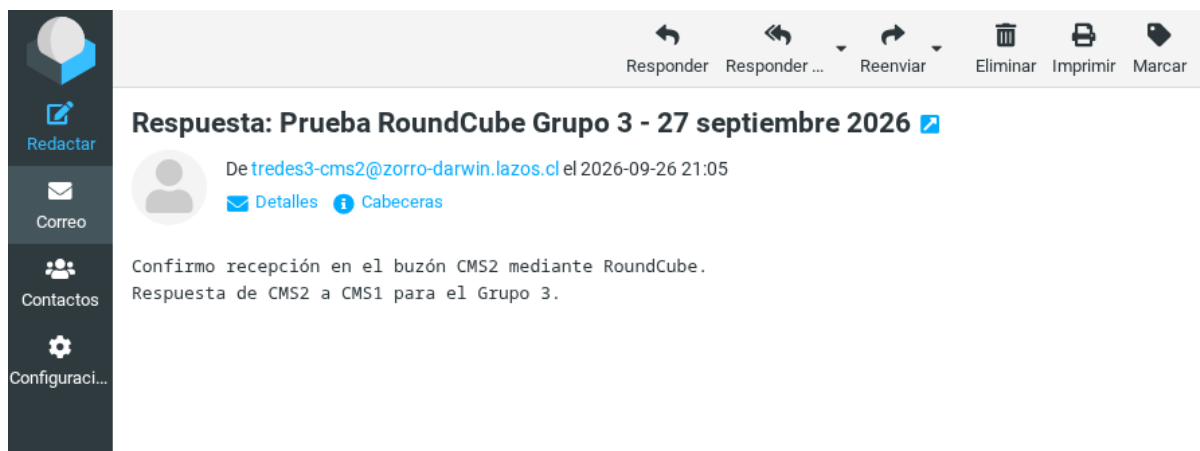


Figura 18: Respuesta de CMS2 abierta en el buzón de CMS1: el envío funciona en ambos sentidos.

Resultado. Correo interno operativo en ambos sentidos con Postfix, Dovecot y RoundCube. Queda fuera de este resultado el envío hacia dominios externos, que no probamos.

3.7. Firewall y SELinux – actividades 9 y 10

Firewall: qué y por qué. El firewall decide qué conexiones entrantes llegan a cada servicio. Dejamos abiertos solo los puertos que cada servicio necesita, y la Tabla 5 explica la razón de cada uno. No abrimos MariaDB (3306), porque solo escucha en loopback, ni IMAP sin cifrar (143), porque RoundCube consulta a Dovecot desde el mismo VPS. Tampoco abrimos POP3 ni HTTPS, porque no están implementados. Las reglas se aplicaron tanto en runtime como de forma permanente.

Cuadro 5: Puertos publicados y su justificación.

Puerto / protocolo	Servicio	Justificación
22/TCP	SSH	Administración remota.
53/TCP y UDP	PowerDNS	Consultas DNS autoritativas (UDP el uso normal, TCP para respuestas grandes).
80/TCP	Apache	Sitios web, PowerAdmin y RoundCube, distinguidos por <code>Host</code> .
25/TCP	Postfix	Recepción de correo por SMTP.
993/TCP	Dovecot	Acceso IMAP cifrado desde el exterior.
2121/TCP	vsftpd	Canal de control FTP alternativo al 21, bloqueado en la red.
30000–30100/TCP	vsftpd	Canal de datos del modo pasivo.
9000/TCP	Chat	Aplicación cliente-servidor en Python.

```
$ firewall-cmd --list-services; firewall-cmd --list-ports
dhcpv6-client http ssh
25/tcp 53/tcp 993/tcp 2121/tcp 9000/tcp 30000-30100/tcp 53/udp
```

Figura 19: Servicios y puertos permitidos en firewalld. La escucha local de MariaDB y los controles SELinux se registran por separado en las evidencias.

La Figura 19 muestra la salida de firewalld con esos puertos.

SELinux: cómo lo tratamos. SELinux es una segunda capa de seguridad que limita lo que cada proceso puede hacer, aunque los permisos Unix se lo permitan. Nunca lo desactivamos: se mantuvo en **Enforcing** durante todo el taller. Cada vez que algo fue bloqueado, revisamos qué operación se denegaba y aplicamos la solución más estrecha posible:

- **Booleanos activados:** `httpd_enable_homedirs`, `httpd_can_network_connect_db` y `ftpd_use_passive_mode`.
- **Booleanos que dejamos apagados:** `httpd_can_network_connect`, `ftpd_full_access` y `ftpd_anon_write`, porque otorgarían permisos mucho más amplios de lo necesario.
- **Etiquetas de archivo y de puerto** específicas para las rutas modificables y los puertos de FTP.
- **Dos módulos locales acotados:** `ftpd_httpd_content` (para el error 553 de FTP) y `roundcube_http_mail` (para que RoundCube conecte a IMAP y SMTP).

Las pruebas de FTP y de correo se hicieron con `getenforce` devolviendo **Enforcing**, así que los flujos probados funcionan bajo la política activa. Esto no equivale a certificar que no exista ninguna otra denegación posible en rutas que no probamos.

3.8. Aplicación de sockets – actividad 11

Qué hicimos. Desplegamos el servidor de chat en Python 3, escuchando en TCP/9000, y lo ejecutamos como servicio systemd (`chat-texto.service`) bajo la cuenta `tresdes3`. Usar systemd permite que el servidor siga disponible sin mantener una sesión SSH abierta.

Adaptaciones y por qué. El programa original esperaba entrada interactiva por consola. Para que funcionara como servicio, la unidad le entrega esa entrada mediante un FIFO privado (`/run/`

`chat-texto/chat.stdin`). Además, al probarlo vimos que tras la primera desconexión el servidor no liberaba la sesión y no atendía a un segundo cliente, así que modificamos `servidor.py` para que libere la sesión y acepte la siguiente sin reiniciar el servicio. Por eso la versión desplegada deriva del programa del taller pero no es idéntica al archivo original; guardamos respaldo de ambas. El cliente original, más un `conectar.py` para indicar servidor y puerto, se entrega en `entrega/chat-texto/`, junto con las instrucciones de ejecución.

Cómo lo probamos. El protocolo es TCP: el cliente se conecta a `200.13.5.39:9000`, envía texto y recibe la respuesta del servidor (Figura 21). Confirmamos que el puerto estaba en `LISTEN`, que `firewalld` lo permitía y ejecutamos el cliente desde el equipo local. La Figura 20 muestra dos conversaciones consecutivas: la segunda se hizo después de cerrar la primera y sin reiniciar el servicio, lo que demuestra tanto la comunicación de ida y vuelta como la corrección del problema anterior. La salida distingue el texto enviado de la respuesta rotulada **Servidor**, y los números de sesión permiten asociar cada respuesta a su conexión. El journal de la unidad registra las mismas dos sesiones.

```

Conexion consecutiva 1
Conectado al servidor 200.13.5.39:9000
Mensaje de prueba local G3 - sesion 1
Servidor: Respuesta VPS G3 - sesion 1
^C
Cliente finalizado por el usuario.
Conexion consecutiva 2
Conectado al servidor 200.13.5.39:9000
Mensaje de prueba local G3 - sesion 2
Servidor: Respuesta VPS G3 - sesion 2
^C
Cliente finalizado por el usuario.
OK: dos conversaciones consecutivas sin reiniciar servicio.

```

Figura 20: Cliente Python desde el equipo local: dos intercambios consecutivos con el servidor TCP/9000. Mensajes y respuestas coinciden por número de sesión y el servicio no se reinició entre ellas.

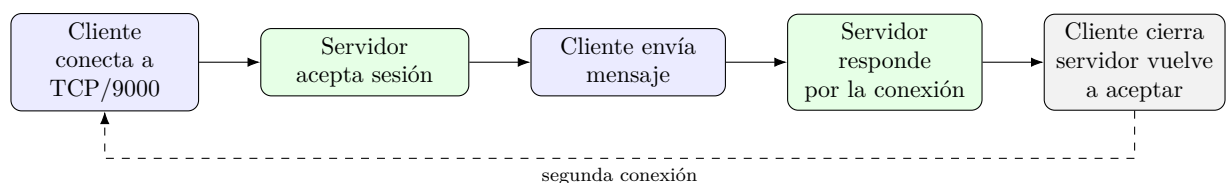


Figura 21: Ciclo de conexión: dos intercambios consecutivos y aceptación de una nueva conexión sin reiniciar la unidad.

Resultado. El servidor atiende varias sesiones consecutivas desde un cliente externo, con el puerto abierto en el firewall y ejecución persistente.

4. Verificación integrada – actividad 12

La Tabla 6 reúne el resultado de cada prueba con su evidencia. Un detalle de honestidad sobre las fechas: combina la ronda funcional del 26 de septiembre por la noche (27 en el VPS, que usa la zona horaria Europe/Madrid) con el lote de DNS y PowerAdmin repetido el 28 de septiembre desde Chile. “Correcto” se refiere al alcance observado en cada fila; no significa que todo se repitiera el mismo día.

Cuadro 6: Resultados de la verificación integrada.

Área	Prueba y resultado obtenido	Estado	Evidencia
VPS	AlmaLinux 9.8, IP 200.13.5.39, línea base previa y SELinux Enforcing.	Correcto	Figuras 1, 2
Usuarios	Tres cuentas y tres <code>public_html</code> individuales, modo 755.	Correcto	Figura 13
DNS / BD	A de la raíz y seis nombres; MX 10; NS/SOA; UDP y TCP autoritativos; zona de 11 registros; BD solo en loop-back.	Correcto	Figuras 5, 6
PowerAdmin	Alta de un TXT temporal, consulta por DNS, borrado y zona final de 11 registros.	Correcto	Figuras 7, 8
Web / CMS	HTTP 200 y contenido propio en WordPress, Joomla y Grav.	Correcto	Figuras 10–12
FTP	Tres usuarios: login, modificación, HTTP idéntico, jaula; anónimo rechazado (530).	Correcto	Figura 15
Correo	CMS1 envió; CMS2 leyó y respondió; CMS1 leyó la respuesta.	Correcto	Figuras 17, 18
Firewall	Reglas runtime y permanentes; puertos de cada servicio; BD solo local.	Correcto	Figuras 6, 19
SELinux	Enforcing ; contextos y módulos específicos; flujos FTP y correo funcionales.	Correcto	Figuras 15, 17, 19
Sockets	Dos conversaciones desde un cliente local sin reiniciar el servidor.	Correcto	Figura 20

5. Dificultades encontradas y cómo las resolvimos

La Tabla 7 reúne los problemas que afectaron una prueba concreta. Para cada uno indicamos cómo se manifestó, cómo lo investigamos, cuál era la causa y qué resultado tuvo la solución.

Cuadro 7: Dificultades: síntoma, diagnóstico, solución y resultado.

Síntoma	Diagnóstico (herramientas y causa)	Solución y resultado
PowerDNS no iniciaba.	<code>systemctl</code> , permisos del archivo y <code>pdnsutil</code> : el proceso no podía leer su configuración.	Grupo <code>pdns</code> y modo 640 en el archivo. El servicio inició; <code>pdnsutil</code> validó la zona y <code>dig</code> recibió respuestas.
Falta el A de la raíz aunque la zona “validaba”.	<code>pdnsutil</code> y <code>dig</code> : el chequeo revisa sintaxis, no que falte un registro.	Agregamos el A de la raíz; zona final de 11 registros sin errores.
Joomla devolvía HTTP 500.	Log de PHP, memoria y contextos SELinux: memoria insuficiente y caché sin etiqueta de escritura.	Límite de 512 MiB y etiqueta solo en las rutas necesarias. Portada y administración volvieron a HTTP 200.

Síntoma	Diagnóstico (herramientas y causa)	Solución y resultado
PowerAdmin mostraba error tras instalar. TXT de prueba parecía ausente o persistente.	Navegador y revisión de archivos: el directorio del instalador seguía presente. PowerAdmin, SQL y dig: la caché de PowerDNS conservaba la respuesta anterior.	Lo movimos al respaldo; luego se pudo crear, consultar y borrar un TXT. Purgamos solo ese nombre con <code>pdns_control</code> ; el alta y la baja se vieron por DNS.
FTP: login correcto pero STOR devolvía 553.	<code>stat</code> , <code>ls -Z</code> y <code>sesearch</code> : los permisos Unix permitían escribir, pero SELinux impedía a <code>ftpd_t</code> modificar el tipo web.	Módulo local acotado. Las tres cuentas hicieron dos escrituras (226) y HTTP mostró la nueva versión, con SELinux en Enforcing.
RoundCube entraba pero no enviaba.	Registro de RoundCube: intentaba el puerto 587, pero Postfix escuchaba en el 25.	<code>smtp_host</code> fijado en <code>127.0.0.1:25</code> . CMS2 abrió el mensaje de CMS1 y CMS1 la respuesta.
Chat atendía una sesión y luego quedaba ocupado.	Segundo cliente y journal de <code>chat-texto</code> : el servidor no liberaba la conexión anterior.	Adaptamos el ciclo de aceptación. Dos conversaciones seguidas obtuvieron respuesta sin reiniciar.

6. Análisis de resultados y limitaciones

Qué nos enseñó el trabajo. Casi todos los problemas se resolvieron cuando dejamos de mirar solo el estado del servicio y probamos el recorrido completo. La zona de PowerDNS estaba “válida” aun faltando el A de la raíz; el chat estaba “activo” pero rechazaba una segunda conexión; RoundCube cargaba pero enviaba a un puerto donde nadie escuchaba. En los tres casos, mirar solo el estado habría ocultado la falla. También aprendimos que, en un sistema con SELinux, los permisos Unix por sí solos no bastan: la falla de FTP (error 553) fue la mejor muestra de ello.

Aislamiento y alcance de las pruebas. Los tres sitios comparten IP, pero Apache separa los sitios por el encabezado `Host` y `vsftpd` separa a los usuarios por su identidad local y su jaula. La comparación de contenido tras subir un archivo por FTP mostró que cada transferencia terminó en el sitio de su propio `VirtualHost`, y el rechazo de rutas ajenas y del usuario anónimo comprobó los límites que ensayamos. No afirmamos que cualquier ruta imaginable esté aislada, sino que la separación entre las tres cuentas quedó demostrada en los casos registrados. Las capturas de RoundCube corresponden a un navegador real controlado por automatización y la prueba FTP usó un cliente automatizado; lo indicamos para no atribuir las a una operación manual.

Limitaciones del despliegue.

- **Sin cifrado en web y FTP:** los sitios y el webmail funcionan por HTTP y el FTP transmite sin cifrar. Es aceptable en el entorno del taller; en producción habría que usar HTTPS y FTPS o SFTP.
- **Certificado de correo autofirmado:** no es de confianza pública.
- **POP3 no implementado:** solo IMAP e IMAPS.

- **Cuenta admin@:** estaba prevista como buzón adicional y no la comprobamos; no forma parte del flujo mínimo confirmado.

Respaldamos la configuración y la base antes de editar, y las copias con datos sensibles quedaron fuera del repositorio. El video (mudo, con su guion) se entrega por separado.

7. Conclusiones

Logramos dejar en el VPS una infraestructura funcional para Fauna Araucanía: una zona DNS autoritativa verificada desde fuera, administrable con PowerAdmin; tres sitios con tres CMS distintos y contenido propio; FTP en puerto alternativo con usuarios aislados; correo que se envía y se recibe en ambos sentidos; y un servidor de sockets que atiende sesiones consecutivas. Todo esto se probó con SELinux en **Enforcing** y con el firewall abierto solo en los puertos necesarios.

Los errores iniciales nos obligaron a entender cómo se conectan los servicios (DNS, Apache, permisos, SELinux, firewall y aplicaciones), y esa comprensión fue tan valiosa como la instalación en sí. Para un servicio público de uso continuo, lo primero sería cifrar el acceso web y FTP y reemplazar el certificado de correo por uno confiable.

Repositorio del proyecto: <https://github.com/Luchosqi/taller-redes-grupo-3>.

8. Referencias y extractos de evidencia

8.1. Referencias

1. Enunciado del Taller de Redes y actividades 1–12 entregados por el curso.
2. PowerDNS, *Generic SQL Backends* y *pdnsutil*: <https://doc.powerdns.com/authoritative/backends/generic-sql.html> y <https://doc.powerdns.com/authoritative/manpages/pdnsutil.1.html>. Consultados el 28-09-2026.
3. Red Hat, *Using SELinux* y *Configuring firewalls and packet filters* (RHEL 9): https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/using_selinux/ y https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_firewalls_and_packet_filters/. Consultados el 28-09-2026.
4. Postfix, *Basic Configuration*: https://www.postfix.org/BASIC_CONFIGURATION_README.html; Python, módulo `socket`: <https://docs.python.org/3/library/socket.html>. Consultados el 28-09-2026.
5. Evidencias del grupo: capturas y transcripciones en `evidencias/10-revision/`, `evidencias/11-validacion-final/` y `evidencias/12-auditoria-lotel-dns-poweradmin/`. Código del chat en `entrega/chat-texto/`; política FTP en `configuraciones/selinux/`.

8.2. Extractos de las evidencias guardadas

Estas líneas provienen de los archivos de evidencia del repositorio. No son la salida íntegra de los archivos de configuración.

Correo (`evidencias/06-correo/validacion-postfix-dovecot-roundcube.txt`):

```
Postfix 3.5.25 activo, SMTP en TCP/25; postfix check sin errores. Dovecot 2.3.16 activo con IMAP e IMAPS; buzones Maildir locales para CMS1 y CMS2. RoundCube 1.7.4 desplegado en webmail; respuesta HTTP 200 y título "Grupo 3 - Webmail".
```

Usuarios y permisos (evidencias/02-usuarios/usuarios-directorios.txt):

```
tredes3-cms1:x:1001:1001::/home/tredes3-cms1:/bin/bash
root:root 755 /home/tredes3-cms1
tredes3-cms1:tredes3-cms1 755 /home/tredes3-cms1/public_html
```

Firewall y SELinux (evidencias/07-firewall-selinux/validacion-firewall-selinux.txt): el firewall publicó TCP/25, TCP/993, TCP/2121, TCP/9000 y TCP/30000–30100; `getenforce` devolvió `Enforcing`; `ftpd_full_access`, `ftpd_anon_write` y `httpd_can_network_connect` quedaron apagados.

A. Parámetros de configuración que determinan el comportamiento

No reproducimos los archivos de configuración completos porque incluyen valores sensibles; en su lugar, la Tabla 8 muestra los parámetros que cambian el comportamiento observable, el motivo de cada uno y la prueba que lo verifica.

Cuadro 8: Parámetros clave de cada servicio y su motivo.

Componente	Parámetro o ubicación	Motivo y verificación
MariaDB	<code>bind-address=127.0.0.1;</code> <code>bases pdns_redes3, web1_db, web2_db</code>	No expone el puerto 3306 en la interfaz pública; <code>ss</code> confirma la escucha en loopback.
PowerDNS	<code>launch=gmysql;</code> <code>gmysql-host=127.0.0.1;</code> <code>base pdns_redes3</code>	Lee la zona desde MariaDB; <code>pdnsutil</code> y <code>dig</code> contrastan lo almacenado con lo respondido.
Apache	<code>VirtualHosts</code> en <code>cms-sites.conf,</code> con <code>DocumentRoot</code> en cada <code>public_html</code>	El <code>Host</code> elige un sitio distinto en la misma IP; <code>httpd -S</code> y las tres portadas lo verifican.
PHP-FPM	<code>memory_limit=512M;</code> pool de máximo seis procesos	Corrige el agotamiento de memoria de Joomla sin ampliar permisos; portada y administración vuelven a HTTP 200.
vsftpd	<code>listen_port=2121;</code> <code>pasv_min_port=30000;</code> <code>pasv_max_port=30100</code>	Control fuera del puerto 21 y datos pasivos alcanzables; las tres cuentas completan STOR con 226.
vsftpd	<code>anonymous_enable=NO;</code> <code>chroot_local_user=YES;</code> <code>local_root=/home/\$USER</code>	Limita identidades y ruta visible; anónimo recibe 530 y rutas ajenas 550.
Postfix	<code>Dominio</code> local en <code>mydestination;</code> buzón <code>Maildir/</code>	Entrega a cuentas Unix existentes; las dos INBOX contienen los mensajes enviados.
RoundCube	<code>smtp_host=127.0.0.1:25</code>	Apunta al puerto donde realmente escucha Postfix; desaparece el rechazo de conexión.
Chat	<code>chat-texto.service;</code> FIFO <code>/run/chat-texto/chat.</code> <code>stdin;</code> TCP/9000	Mantiene el servidor activo fuera de SSH; el cliente recibe respuestas en dos sesiones seguidas.

Estas decisiones dependen unas de otras. El FTP pasivo necesita puertos abiertos en firewalld, etiquetas SELinux de puerto y permisos de escritura en el árbol web; el correo necesita MX y A correctos, Postfix para entregar, Dovecot para leer y una configuración SMTP de RoundCube que apunte al puerto real; y los CMS necesitan resolución de nombres, VirtualHosts, PHP-FPM y acceso a su persistencia. Por eso, cada vez que corregimos una falla compartida, volvimos a probar los demás servicios.